

Clase 14: Simulación

Profesor: Pablo J Piccoli Bornia

Auxiliares: Daniela Longas M Victoria Tolosa

Montecarlo

Simulación de Montecarlo

Cuando obtener resultados exactos es complicado, la simulación Montecarlo permite estimarlos a partir de experimentos aleatorios en contextos con incertidumbre o variabilidad.

- ▶ Repetimos muchas veces un experimento aleatorio.
- ▶ Registramos los resultados obtenidos.
- ▶ Calculamos promedios, probabilidades u otras medidas de interés.

Idea clave: cuando no podemos calcular una solución exacta, podemos simular muchas veces y observar el comportamiento del sistema.

Simulación de Montecarlo

Ejemplo de esto podría ser lanzar un dado de 6 caras.

Nosotros sabemos que la probabilidad de que toque un número en particular al arrojarlo es de $\frac{1}{6}$. Ahora bien, si nos dicen que un lado está trucado? Cómo haríamos para saber cuál es ese lado en cuestión.

Idea: Tirar el dado 1000 veces y dividir por mil la cantidad de de veces que tocó dicho lado. Para el número cuyo resultado sea mayor a $\frac{1}{6}$, ese es el lado trucado.

Simulación de Montecarlo

Nosotros podemos tirar 1K veces un dado?

Si, pero es sumamente impráctico. La computación ya resolvió esto por nosotros mediante un concepto visto en clases anteriores: la recursión.

Podemos también simular el comportamiento aleatorio con una librería ya vista en la materia: random

Comandos librería random

Algunas funciones útiles:

► `random.randint(a, b)`

Genera un número entero al azar entre a y b (*ambos inclusive*). Ejemplo:

```
random.randint(1, 6)    # un número entero entre 1 y 6
```

► `random.choice(lista)`

Devuelve un elemento elegido al azar de una lista. Ejemplo:

```
random.choice(['cara', 'ceca'])    # simula una moneda
```

► `random.random()`

Devuelve un número decimal entre 0.0 y 1.0. Ejemplo:

```
random.random()    # 0.3746, 0.9211, etc.
```

Concepto: semilla

Una semilla (seed) es un número que suele ser input de un generador de resultados random. Si cargamos una semilla a una instancia de python, obtendremos siempre los mismos resultados “random”.

Es una manera de generar resultados aleatorios pro replicables

```
import random

random.seed(42)          # fija la semilla

print(random.randint(1,6))
print(random.randint(1,6))
```

Ej 1

Ejercicio: cartas numeradas

Ejercicio 1.

Queremos estimar la probabilidad de que **al menos una de dos cartas sacadas de un mazo numerado del 1 al 10 sea mayor o igual a 8.**

Se pide escribir un programa en Python que:

1. Simule la extracción de **2 cartas con reemplazo** usando `random.randint(1,10)`.
2. Repita la simulación **10.000 veces**.
3. Calcule la proporción de veces que al menos una carta es ≥ 8 .
4. Muestre la probabilidad estimada por consola.

La probabilidad exacta es 0,51. ¿El programa obtuvo una buena aproximación?

Ej 2

Simulación: pedidos en un restaurante

Ejercicio 2.

En un restaurante, cada cliente elige su comida al azar entre cuatro opciones: 'pizza', 'empanadas', 'ensalada' y 'milanesa'.

Se atienden **3 clientes** (independientes) y queremos estimar, mediante simulación, la probabilidad de que **al menos uno** pida pizza.

Para ello:

1. Representar las opciones como una lista: `menu = ['pizza', 'empanadas', 'ensalada', 'milanesa']`.
2. Repetir el experimento 10000 veces.
3. En cada simulación, elegir 3 comidas al azar usando `random.choice(menu)`.
4. Contar cuántas veces aparece al menos una 'pizza'.
5. Estimar la probabilidad como:

$$\text{probabilidad estimada} = \frac{\text{número de simulaciones con al menos una pizza}}{\text{número total de simulaciones}}$$

La probabilidad exacta es 0,578125. ¿El programa obtuvo una buena aproximación?

Variables continuas

Cuando queremos modelar casos discretos (arrojar un dado) `random.randint(a,b)` is the way to go.

Si queremos modelar algo que varía de manera continua podemos usar `random.random()` o su generalización `random.uniform(a,b)`

La función `random.uniform(a, b)` es la generalización de `random.random()`: genera un número real en cualquier intervalo $[a, b]$.

```
y = random.uniform(5, 10)
print(y)    # Ejemplo: 7.4832...
```

Ej 3

Ejercicio 3.

Supongamos que la temperatura diaria en una ciudad durante el verano varía uniformemente entre 20°C y 35°C. Queremos estimar la probabilidad de que un día haga **menos de 25°C**.

Para ello podemos usar Montecarlo:

1. Simular n días usando `random.uniform(20, 35)`.
2. Contar cuántos días tuvieron temperatura menor a 25°C.
3. Estimar la probabilidad como:

$$\text{probabilidad estimada} = \frac{\text{número de días con temp menor a 25°C}}{\text{número total de días simulados}}$$

La probabilidad exacta es 0,33333 (periódico). ¿El programa obtuvo una buena aproximación?

Experimentos con memoria/duración variable

Ciertos experimentos no tienen una cantidad N fija de veces en la cual son ejecutados, si no que dependen de que suceda cierto evento.

Ej4:

Calcular la cantidad promedio de tiradas que uno debe hacer de un dado hasta que toca el número 6.

De la misma forma, hay ciertos experimentos que tienen “memoria” y deben guardar resultados intermedios para poder ejecutarse

Ej5:

Calcular la cantidad promedio de veces que hay que tirar un dado de manera de que salgan 2 caras consecutivas.